

Manual Mentor Payout without Razorpay Payout

A complete design for settling mentor earnings from the admin panel using company UPI / IMPS / NEFT — covering the mobile app, web portal, and admin panel. Razorpay remains the learner collection rail only.

Document type	Architecture, operations flow, and implementation plan
Date	21 August 2026
Audience	Product, engineering, finance / operations
Classification	Internal
Mobile app	connectfront — React Native (Android / iOS)
Web portal	connectiqo_web-portal — Next.js
Admin panel	adminsidedconnectfornt/admin — Vite + React
Backend	Supabase (Postgres, RLS, Edge Functions)
Inbound payments	Razorpay Checkout — keep as-is
Outbound payouts	Manual settlement — recommended
Minimum withdrawal	₹5,000 (already enforced in app and web)

Decision in one sentence

Do not wire RazorpayX or Razorpay Route for mentor settlement. Treat the mentor wallet as an internal ledger, let mentors request a withdrawal against a saved UPI ID, and have an operator pay from the company account and record the UTR in admin. The request path already exists in TEST MODE; what is missing is a safe admin fulfilment workflow.

00

Contents

1. Executive summary
2. Objective and non-goals
3. Systems in scope
4. How money moves today
5. Code and data map
6. Findings and risks
7. Options considered
8. Recommended architecture
9. End-to-end operating flow (flowcharts)
10. Withdrawal status machine
11. Data model and RPCs
12. Admin panel specification
13. Mobile app changes
14. Web portal changes
15. Notifications and mentor experience
16. Security, audit, and India compliance
17. Implementation phases
18. Test plan
19. What not to do
20. Appendix — current schema and file index

How to read this document

Sections 1–6 describe the current product. Sections 7–10 recommend the operating model. Sections 11–15 are the build spec. Sections 16–18 are rollout, risk, and QA. Implementers can start at section 11; operators can start at section 9.

01 Executive summary

Connectiqo already collects learner payments through Razorpay and credits an internal mentor wallet after a session is marked completed. Mentors can request a withdrawal from the mobile Wallet screen and the web Earnings page. That request does not send money through Razorpay today. The edge function `process-withdrawal` is explicitly in TEST MODE: it deducts the wallet and inserts a row into `withdrawal_requests` with status pending, returning an ID of the form `manual_{id}`.

The intended live path in comments is RazorpayX (Razorpay Payouts). That is exactly what this programme should not enable. RazorpayX requires a funded RazorpayX current account, contact/fund-account KYC, webhook reconciliation, and extra commercial terms. The product already has the pieces of a simpler model: UPI on the mentor profile, a request queue, and an admin Payments screen.

The gap is operational, not conceptual. Admin can open `withdrawal_requests` as generic CRUD. There is no dedicated payout queue, no Copy-UPI / Copy-amount action, no UTR field, no atomic complete/reject that restores the wallet, and no push to the mentor when money is sent. Worse, a reject performed by editing the row in CRUD would leave the deducted balance unrestored.

The recommended operating model is:

- 1 Keep Razorpay Checkout for learners (booking and video unlocks).
- 2 Stop treating Razorpay Route linked-account KYC as a prerequisite for withdrawals.
- 3 Let mentors save a UPI ID (and optionally a bank account) as payout details.
- 4 On withdraw, hold funds on the internal wallet and create a pending request.
- 5 An admin operator pays from the company bank / UPI app (GPay, PhonePe, IMPS, NEFT).
- 6 Admin records method + UTR / UPI reference and marks the request completed.
- 7 On reject, restore the wallet atomically and notify the mentor.

Dimension	Today	Target
Learner collection	Razorpay order + verify	Unchanged
Mentor earning credit	<code>complete_session_payment</code> RPC after session complete	Unchanged (internal ledger)
Instant split to mentor	Optional Razorpay Route transfer	Disable / leave unused
Payout setup	Razorpay Route linked account + KYC email	Save UPI (optional bank) only
Withdrawal request	<code>process-withdrawal</code> TEST MODE	Keep request; never call RazorpayX
Fulfilment	Generic CRUD on <code>withdrawal_requests</code>	Dedicated Admin Payout Control Panel
Proof of payment	<code>razorpay_payout_id</code> unused	UTR / UPI ref + method + paid_at + operator

Table 1 — Current vs target settlement model.

02 Objective and non-goals

Objective

Enable Connectiqo operations to pay mentors from the admin panel, for both the mobile app and the web portal, without using Razorpay Payout (RazorpayX) or Razorpay Route transfers as the settlement rail.

In scope

- Mentor payout identity (UPI, optional bank).

- Withdrawal request lifecycle (pending → processing → completed | rejected).
- Admin fulfilment UI with proof-of-payment fields.
- Wallet hold / restore so a reject cannot strand funds.
- Parity of status display on mobile Wallet and web Earnings.
- Push / in-app notification when a payout is sent or rejected.
- Audit trail (who paid, when, UTR, notes).

Out of scope (intentionally unchanged)

- Learner checkout on Razorpay (create-razorpay-order, verify-razorpay-payment).
- Play Billing / Apple IAP for in-app video unlocks.
- Platform fee and GST calculation on the booking.
- Changing the ₹5,000 minimum unless product later revises it.
- Automated bank-file / NACH bulk payout (a later phase if volume grows).
- TDS computation engine (record a note/field; do not auto-deduct in v1 unless finance requires it).

Important distinction

Razorpay stays. Only the outbound mentor settlement is taken off Razorpay. Learners still pay Connectiqo; Connectiqo then settles mentors from its own bank / UPI, the same way many early marketplaces run until payout volume justifies a PSP.

03 Systems in scope

Surface	Path	Role in payout
Mobile app	C:\Flutter\Freelancing\Project\connectfront	Mentor saves UPI, views wallet, requests withdrawal, sees request status.
Web portal	D:\connectiqo\connectiqo web\connectiqo web\connectiqo_web-portal	Same mentor flows as the app, already lists withdrawal history.
Admin panel	C:\Flutter\Freelancing\Project\admin\adminsideconnectfornt\admin	Operator queue: pay outside the product, then mark paid / reject with UTR.
Supabase	connectfront/supabase (shared backend)	Tables, RPCs, Edge Functions. App and web already call the same functions.

Table 2 — Surfaces. One backend; three clients.

Because mobile and web already invoke the same Edge Functions (process-withdrawal, create-linked-account, get-account-status), a backend-first change automatically serves both platforms. Admin is a separate Vite app that talks to the same Postgres with a service-role key.

04 How money moves today

4.1 Learner pays (keep)

- 1 Learner selects slot(s) and pays. App/web call create-razorpay-order.
- 2 Razorpay Checkout collects amount_total_paise (mentor cut + platform fee + GST).
- 3 verify-razorpay-payment verifies the signature and calls claim_and_book_slots.
- 4 That RPC books the slot(s), creates the booking, and inserts earnings with status pending.

4.2 Session completes — wallet credit (keep)

When a booking is marked completed, `bookingApi` runs `complete_session_payment`. That RPC flips the earnings row from pending → completed and upserts `mentor_wallets`: balance and `total_earned` increase by the mentor cut. This is the internal ledger. It is independent of whether rupees have left the company bank account.

4.3 Optional Razorpay Route split (disable for this programme)

Immediately after the RPC, the client fire-and-forgets `transfer-session-payout`. If the mentor has an activated Razorpay Route linked account and the transaction was `route_enabled`, Razorpay splits the mentor cut from the captured payment. If not, the function no-ops with `not_route_enabled` or `account_not_active`. The code comment already says earnings then “stay in the internal wallet ledger for manual withdrawal.” That is the path we should standardise on.

4.4 Mentor payout setup (too heavy for manual settlement)

`PayoutSetupScreen` (app) and `/settings/payout` (web) call `create-linked-account`, which creates a Razorpay Route v2 account, emails the mentor for KYC, and stores `razorpay_account_id`. `get-account-status` polls Razorpay and maps `activated` → `active`. The Wallet screen treats `payoutReady` as KYC active, while the actual Withdraw button only requires a stored UPI and ₹5,000 balance. Those two signals disagree.

4.5 Withdrawal request (already manual)

`process-withdrawal` verifies the JWT, requires `mentor_profiles.upi_id`, calls `deduct_wallet_for_withdrawal`, and inserts `withdrawal_requests` with status pending. It returns `payoutId: manual_{withdrawal.id}`. The file header says: “TEST MODE: Records withdrawal request in DB only. When going live, integrate RazorpayX payout block.” Going live should mean admin fulfilment — not RazorpayX.

4.6 Admin today (unsafe for money movement)

`PaymentsPage` tabs: Transactions, Mentor Wallets, Earnings, Withdrawals. Withdrawals is generic `ResourceCrudPage` on `withdrawal_requests`. Operators can JSON-edit status and `admin_note`. The status filter options are `all` / `pending` / `approved` / `rejected`, but the database CHECK is `pending` | `processing` | `completed` | `rejected`. There is no “approved” value. `TransactionControlPanel` is a better pattern (dedicated status + note editor) but exists only for learner transactions, not payouts.

05

Code and data map

5.1 Primary tables

Table	Purpose	Payout-relevant columns
transactions	Learner payment lifecycle	<code>amount_total_paise</code> , <code>mentor_earning_paise</code> , <code>platform_fee_paise</code> , <code>status</code> , <code>razorpay_*</code>
earnings	Mentor cut per session / video	<code>amount</code> , <code>status</code> (pending completed), <code>source</code> , <code>route_transfer_id</code>
mentor_wallets	Internal available balance	<code>balance</code> , <code>total_earned</code> , <code>total_withdrawn</code>
withdrawal_requests	Payout queue	<code>amount</code> , <code>upi_id</code> , <code>status</code> , <code>admin_note</code> , <code>razorpay_payout_id</code>
mentor_profiles	Payout identity + Route leftovers	<code>upi_id</code> , <code>razorpay_account_id</code> , <code>razorpay_contact_id</code> , <code>razorpay_fund_account_id</code> , <code>kyc_status</code>

Table 3 — Ledger tables. Source: `supabase/migrations`.

5.2 Key files

Layer	File	Notes
App	src/scenes/settings/WalletScreen.js	MIN_WITHDRAWAL = 5000; canWithdraw = balance + UPI; payoutReady = Razorpay KYC active.
App	src/scenes/settings/PayoutSetupScreen.js	Creates Razorpay Route linked account; copy still says “1–2 business days”.
App	src/api/paymentApi.js	requestWithdrawal → process-withdrawal. Comment still says RazorpayX.
Web	src/app/(main)/settings/wallet/page.tsx	Same min and UPI gate; already lists withdrawal_requests.
Web	src/app/(main)/settings/payout/page.tsx	Same linked-account form as the app.
Admin	src/features/payments/PaymentsPage.jsx	KPI cards + CRUD tabs. Filter mismatch on withdrawals.
Edge	supabase/functions/process-withdrawal/index.ts	Manual request writer. TEST MODE. No RazorpayX call.
Edge	supabase/functions/create-linked-account/index.ts	Razorpay Route account create. Not needed for manual payout.
Edge	supabase/functions/get-account-status/index.ts	Polls Razorpay KYC. Blocks “payout ready” UX.
Edge	supabase/functions/transfer-session-payout/index.ts	Route split after session. Should no-op permanently for this programme.

Table 4 — Implementation touchpoints.

5.3 Missing migration (critical)

deduct_wallet_for_withdrawal is called but not in the repo migrations

process-withdrawal invokes RPC deduct_wallet_for_withdrawal. No CREATE FUNCTION for it exists under supabase/migrations (increment_mentor_wallet and complete_session_payment do). If it was applied only in the SQL editor, it is undocumented and may not increment total_withdrawn consistently, or may not exist on a fresh environment. Phase 0 must add this RPC to versioned SQL before any admin fulfilment is built on top of it.

06

Findings and risks

ID	Severity	Finding	Impact if ignored
F1	Critical	Admin CRUD can set status=rejected without restoring wallet.	Mentor loses deducted funds with no bank credit.
F2	Critical	Admin CRUD can set status=completed with no UTR and no operator identity.	Unauditable “we paid you” claims; disputes unresolvable.
F3	Critical	deduct_wallet_for_withdrawal not in versioned migrations.	Staging/prod drift; withdrawals fail or double-count.
F4	High	Wallet deducted at request time; total_withdrawn timing unspecified.	If deducted AND counted as withdrawn while still pending, reports lie.
F5	High	payoutReady requires Razorpay KYC active; withdraw only needs UPI.	Mentors blocked or confused by “Complete payout setup” while UPI exists.

ID	Severity	Finding	Impact if ignored
F6	High	Admin filter uses status “approved”; DB has no such value.	Operators cannot filter the real queue.
F7	Medium	Mobile Wallet does not list withdrawal_requests (web does).	Mentor cannot see pending/processing/rejected on the app.
F8	Medium	No FCM / in-app event when payout is sent or rejected.	Support load: “where is my money?”
F9	Medium	create-linked-account still required mentally for “setup”.	Unnecessary Razorpay KYC emails; Route account sprawl.
F10	Medium	Admin login is localStorage email/password; mutations use service role in the browser.	Existing risk; payouts raise the cost of a leaked admin session.
F11	Low	transfer-session-payout still invoked after every completed session.	Needless Razorpay API calls; confusing route_transfer_id on some mentors.
F12	Low	paymentApi comments still say “Trigger a RazorpayX UPI payout”.	Next engineer re-enables RazorpayX thinking it is unfinished work.

Table 5 — Findings from code review of app, web, admin, and Edge Functions.

F1 is the reason generic CRUD must not remain the fulfilment tool. A status change is a financial event. It has to run inside a single Postgres function that either completes (records proof, bumps total_withdrawn if not already counted) or rejects (restores balance).

07 Options considered

Option	What it is	Verdict
A. RazorpayX Payouts API	Automated UPI/IMPS from a RazorpayX account when mentor requests withdraw.	Rejected — this is the rail we are avoiding (KYC, float, webhooks, extra contract).
B. Razorpay Route instant split	On session complete, transfer mentor cut from the captured payment.	Rejected as primary — requires linked-account KYC; already optional and flaky for launch.
C. Other PSP (Cashfree / Paytm payouts)	Swap RazorpayX for another payout API.	Rejected for v1 — still a vendor onboarding; does not match “manual from admin”.
D. Manual UPI / IMPS / NEFT from company account	Operator pays in GPay/PhonePe/net-banking; records UTR in admin.	Selected — matches current TEST MODE, low vendor lock-in, auditable, works at current volume.
E. Bulk NEFT file (later)	Export pending rows as a bank upload file at end of day.	Phase 3 if daily payout count becomes painful. Same data model.

Table 6 — Alternatives. Option D is the v1 standard.

Option D is not a workaround. It is a legitimate marketplace settlement model: the platform is the merchant of record for the learner, holds the fee, and pays the professional as a payable. The product’s job is ledger accuracy and operator speed (copy UPI, copy amount, paste UTR), not becoming a bank.

08 Recommended architecture

8.1 Principle

Separate collection from settlement. Razorpay is the collection rail. The mentor wallet is a ledger of what Connectiqo owes. A withdrawal request is an accounts-payable ticket. Completing that ticket is an operator action plus a bank proof, not an API call to Razorpay.

8.2 Ledger rules

- Available balance increases only when a session/video earning is completed (existing RPC).
- A withdrawal request immediately holds (decrements) balance so the mentor cannot spend it twice.
- `total_withdrawn` increases only when admin marks the request completed.
- Reject restores balance and never increases `total_withdrawn`.
- Completed and rejected are terminal. No silent re-open from the UI.
- Idempotent: completing an already-completed row is a no-op; never deduct twice.

8.3 Payout identity

Replace “Create Razorpay payout account” with “Save payout details”. Required: UPI ID (validated format: `name@handle`). Optional: account holder name, bank account number, IFSC (for NEFT when UPI fails or amount is large). Do not call Razorpay. Store on `mentor_profiles`. Treat presence of a valid UPI as payout-ready.

8.4 Shared backend, three UIs

App and web remain thin clients of the same Edge Functions. Admin does not update wallet rows by hand. It only calls two RPCs (or one admin Edge Function that wraps them): `admin_complete_withdrawal` and `admin_reject_withdrawal`. `ResourceCrudPage` remains for read-only inspection if needed, but status must be read-only there.

8.5 Operator path (human in the loop)

- 1 Open Admin → Payments → Payout queue (pending first).
- 2 Open a request: mentor name, phone, UPI, amount, wallet snapshot.
- 3 Click Copy UPI and Copy amount; pay from company UPI / net-banking.
- 4 Paste UTR / UPI reference; choose method (UPI, IMPS, NEFT).
- 5 Confirm with a typed-amount check; system marks completed and notifies mentor.
- 6 If UPI invalid or fraud: Reject with reason; wallet restores; mentor is notified.

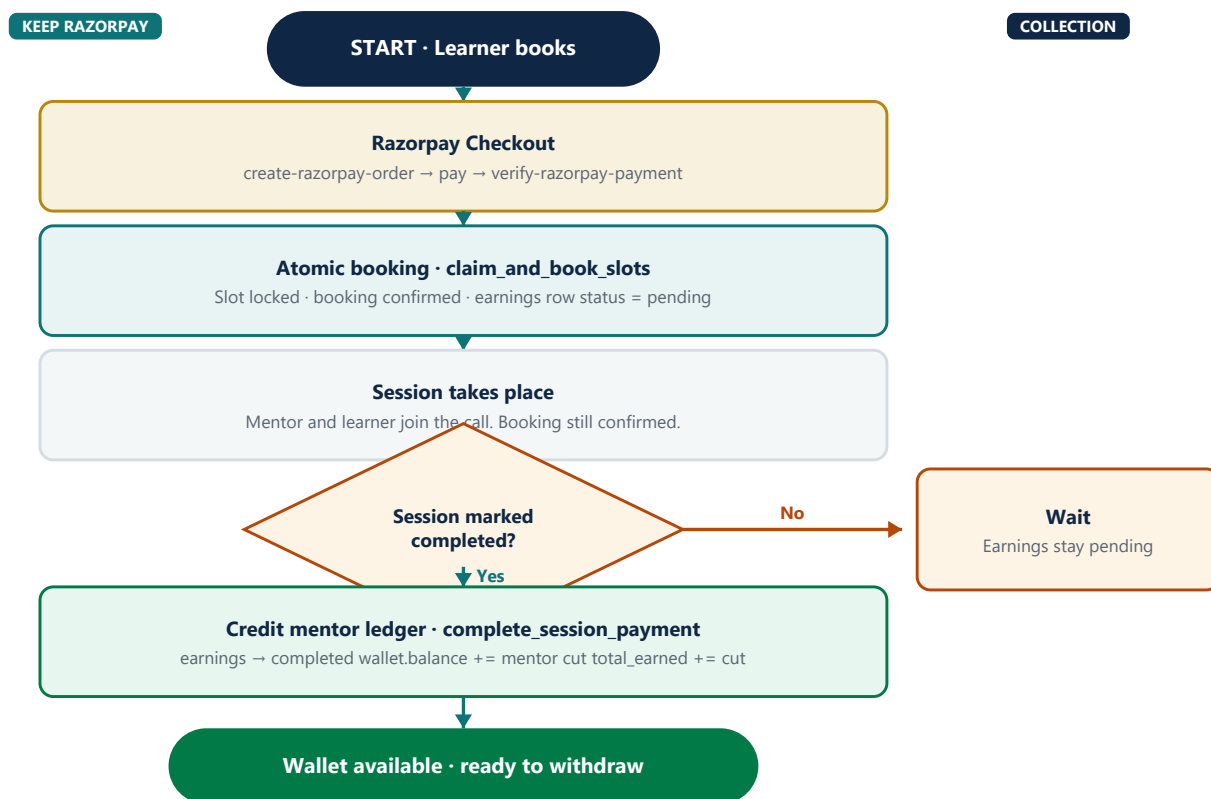
09

End-to-end operating flow

The four figures below are the operating flowcharts. Figure 1 is collection (Razorpay stays). Figure 2 is mentor settlement (no RazorpayX — admin pays, then records the UTR). Figure 3 is the request status machine. Figure 4 is the same journey as swimlanes so each actor’s job is visible.

Figure 1 — Collection to wallet credit

Learner payment is unchanged. Earnings stay pending until the session is completed. Only then does the internal wallet increase. No money has left the company bank yet.



 Razorpay (inbound)
 Backend
 Ledger credit
 Decision / wait

Figure 1 — Razorpay collects; Connectiqo ledger credits the mentor after the session.

09

End-to-end operating flow · continued

Figure 2 — Withdrawal and manual admin payout

This is the flow to implement. The mentor requests a payout; the wallet is held; an operator sends rupees from the company UPI or bank app; success records UTR and completes the row; failure restores the wallet.

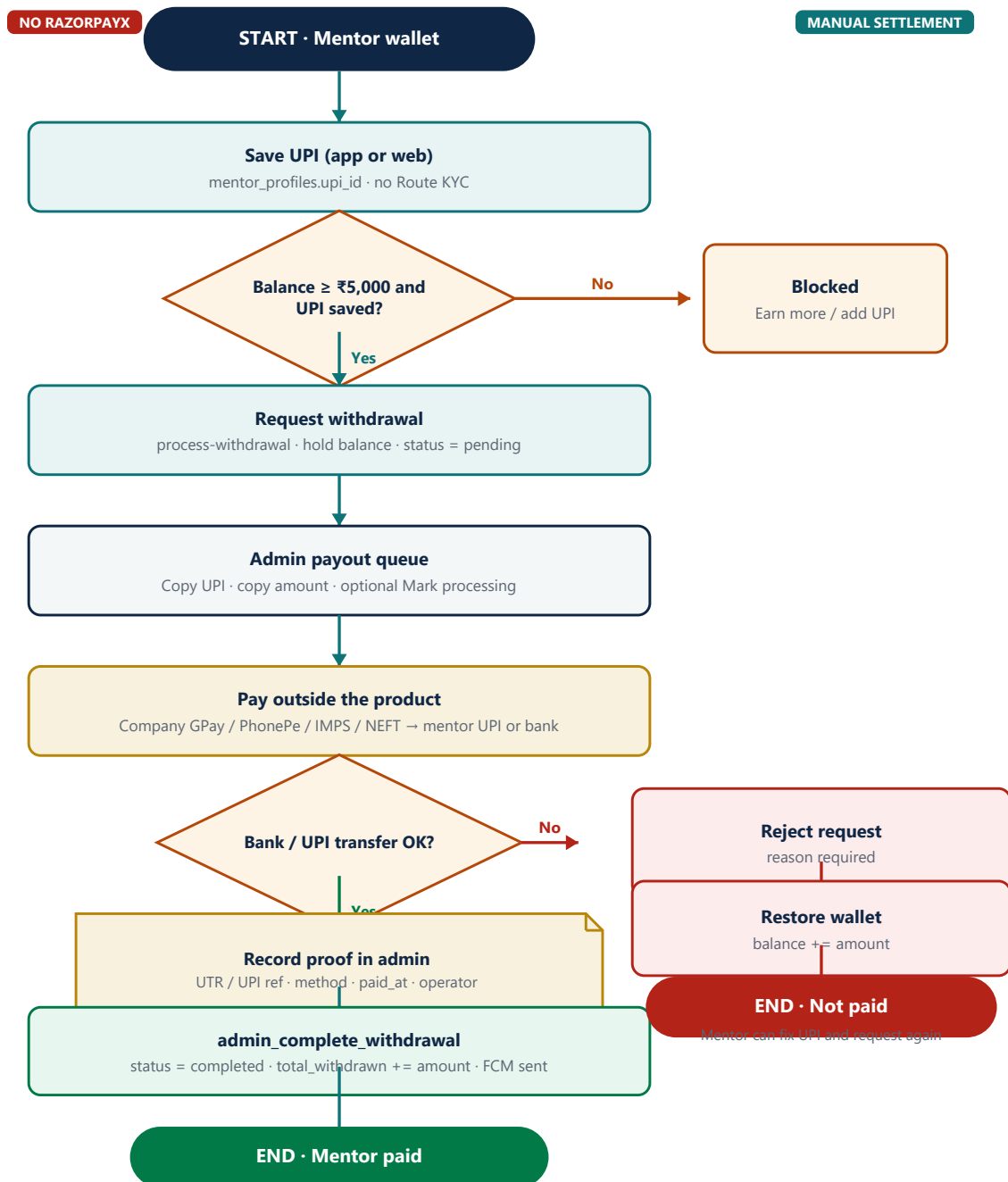


Figure 2 — Manual settlement. Right branch is reject; left branch is paid.

09

End-to-end operating flow · continued

Figure 3 — Status machine

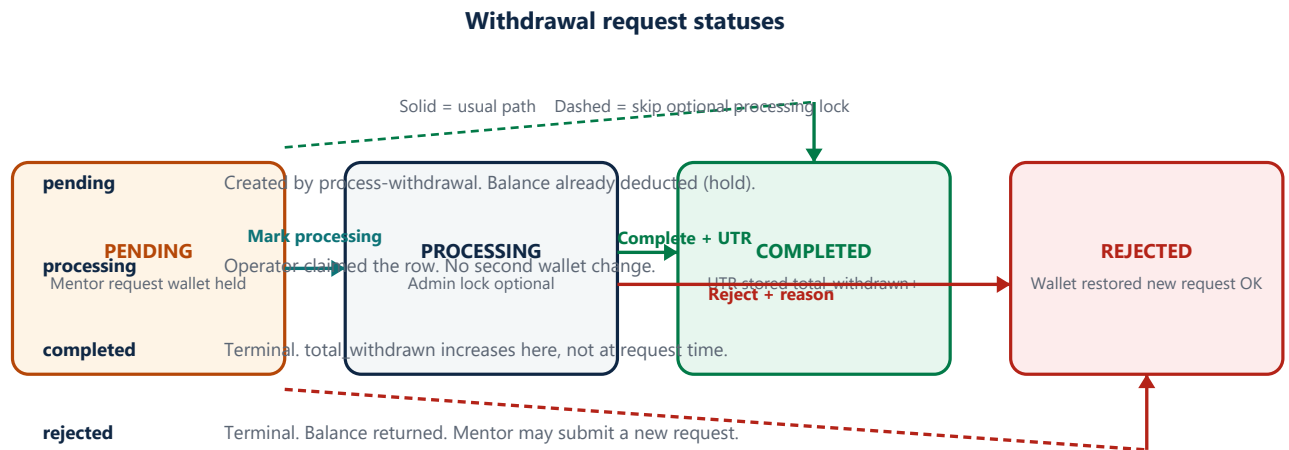
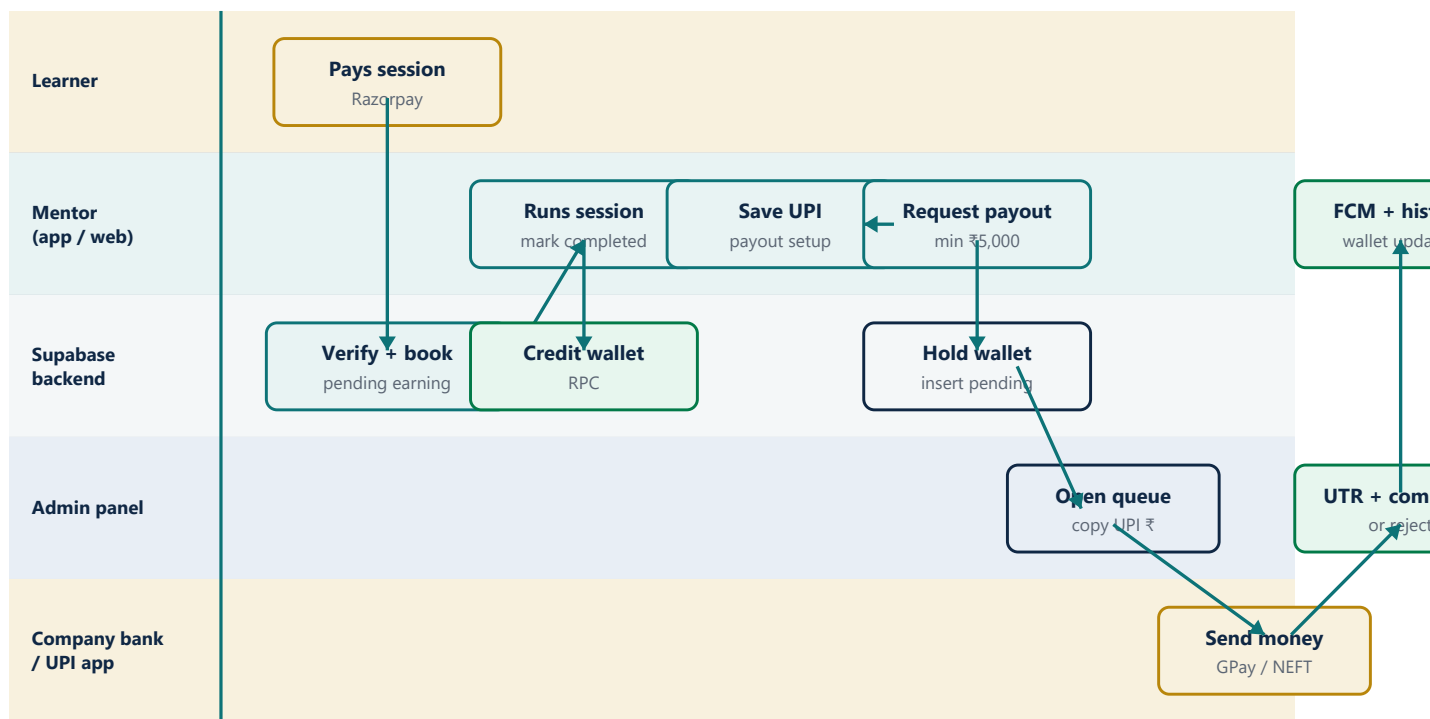


Figure 3 — Allowed status moves. completed and rejected are terminal.

Figure 4 — Swimlane (who does what)

Swimlane · one booking through to mentor payout



Read top to bottom (who), left to right (when). Razorpay is only in the Learner lane.

Figure 4 — Columns are time; rows are actors. Razorpay appears only in the Learner lane.

Happy path — step list (same as Figure 2, left branch)

Step	Name	What happens
1	Learner pays	Razorpay Checkout. Transaction status paid. Unchanged.
2	Session happens	Booking confirmed → call → marked completed.
3	Ledger credit	complete_session_payment: earnings completed; wallet.balance += mentor cut.
4	Mentor saves UPI	Payout setup writes mentor_profiles.upi_id. No Razorpay account.
5	Mentor requests payout	Wallet ≥ ₹5,000. process-withdrawal holds balance, inserts pending request.
6	Admin queue	Payout Control Panel lists pending. Operator sees name, UPI, amount.
7	Admin pays outside app	GPay / PhonePe / IMPS / NEFT from company account to mentor UPI or bank.
8	Admin records proof	admin_complete_withdrawal: UTR, method, paid_at, processed_by. Status completed.
9	Mentor notified	FCM + in-app: “₹X sent to name@upi (UTR ...)”. Wallet shows Withdrawn.

Table 7 — Same happy path as Figure 2, written as a checklist for QA.

Reject path

- Admin marks processing (optional lock so two operators do not pay the same row).
- Payment fails (wrong UPI, beneficiary name mismatch, suspected duplicate).
- admin_reject_withdrawal(p_id, p_reason, p_operator): status=rejected, balance restored, mentor notified.
- Mentor can fix UPI and request again once balance is available.

Double-pay protection

- Only pending (or processing) rows can be completed.
- UPDATE ... WHERE status IN ('pending','processing') RETURNING * — if zero rows, abort.
- UI disables Complete after success; refresh from server.
- Optional: require Mark processing first, so a second admin sees it is in flight.

10 Withdrawal status machine

From	To	Who	Wallet effect	Required fields
(new)	pending	Mentor (process-withdrawal)	Hold: balance – amount	upi_id, amount ≥ 5000
pending	processing	Admin (optional lock)	None	processed_by
pending / processing	completed	Admin complete RPC	total_withdrawn += amount	payout_method, payout_reference (UTR)
pending / processing	rejected	Admin reject RPC	Restore: balance + amount	rejected_reason
completed	—	Terminal	None	No UI action
rejected	—	Terminal	None	Mentor may create a new request

Table 8 — Allowed transitions. “approved” is not a status.

Do not introduce “approved”

The current admin filter already invented an approved status that the CHECK constraint does not allow. Approval is implicit in Complete. If a two-person maker-checker is needed later, add approved as a distinct state before processing — not as a synonym for completed.

11 Data model and RPCs**11.1 Columns to add on withdrawal_requests**

Column	Type	Use
payout_method	TEXT	upi imps neft other
payout_reference	TEXT	UTR / UPI transaction ID (required on complete)
paid_at	TIMESTAMPZ	When the bank/UPI transfer was made
processed_by	TEXT	Admin email / operator id
rejected_reason	TEXT	Shown to mentor on reject

Table 9 — Proof-of-payment fields. razorpay_payout_id can remain unused.

11.2 Optional columns on mentor_profiles

- upi_id — already exists; this becomes the source of truth.
- payout_account_holder TEXT — display name for the operator.
- bank_account TEXT — already conceptually on withdrawal_requests; store on profile too.
- ifsc TEXT — for NEFT fallback.
- Stop writing razorpay_account_id / kyc_status for new mentors. Leave old values untouched.

11.3 RPC: deduct_wallet_for_withdrawal (version in git)

SECURITY DEFINER. Decrement balance only if balance $\geq$ amount. Do not increment total_withdrawn here. Raise on insufficient funds. Return the new balance. Called only from process-withdrawal.

11.4 RPC: admin_complete_withdrawal

Arguments: request id, method, reference, operator, optional note. Lock the row. Require status in (pending, processing). Require non-empty UTR. Set status=completed, paid_at=now(), store proof fields. Increment mentor_wallets.total_withdrawn by the request amount (balance was already held). Return the updated row. Notify via Edge or a trigger.

11.5 RPC: admin_reject_withdrawal

Arguments: request id, reason, operator. Lock the row. Require status in (pending, processing). Set status=rejected. Add amount back to mentor_wallets.balance. Do not touch total_withdrawn. Notify mentor.

11.6 RPC: admin_mark_processing (optional but recommended)

Sets status=processing and processed_by so a second operator sees the lock. No wallet change. Completing or rejecting still allowed from processing.

11.7 Edge Functions

Function	Action
process-withdrawal	Keep. Document as manual queue, not RazorpayX. Continue requiring UPI.
save-payout-details (new) or slim create-linked-account	Auth as mentor. Validate UPI. Update mentor_profiles only. No Razorpay HTTP.
get-account-status	Return { status: upi_id ? 'active' : 'not_started', upild } without calling Razorpay.
transfer-session-payout	Short-circuit to { transferred: false, reason: 'manual_settlement' } or stop invoking it.
notify-payout-status (new)	FCM to mentor: completed or rejected, amount, last-4 of UPI, UTR or reason.

Table 10 — Edge Function changes. Shared by app and web.

11.8 RLS reminder

Mentors can SELECT and INSERT their own withdrawal_requests. They must not UPDATE status. Completion/reject happens via SECURITY DEFINER RPCs invoked with the admin service role. Do not add a broad UPDATE policy for authenticated users.

12 Admin panel specification

Add a dedicated Payout Control Panel on the Payments page, modelled on TransactionControlPanel, shown when the Withdrawals tab is active. Keep the generic CRUD table in read-only / details mode so operators cannot type status=completed in JSON.

12.1 Queue list

- Default filter: pending, then processing. Tabs: Pending · Processing · Completed · Rejected · All.
- Columns: requested at, mentor name, email, UPI, amount (₹), status, age.
- Enrich mentor_id → profiles.name / email / phone (same pattern as listBookingRows).
- KPI strip: pending count, pending rupees, completed this week, rejected this week.
- Copy buttons on UPI and amount (clipboard) — this is the speed feature.

12.2 Detail / fulfil drawer

- Mentor identity, current wallet snapshot (balance, earned, withdrawn), saved UPI, optional bank/IFSC.
- Request amount, created_at, current status.
- Actions: Mark processing · Mark paid · Reject.
- Mark paid fields: payout_method (select), payout_reference (required), paid_at (default now), admin_note.
- Guard: operator must re-type the amount (e.g. 5000) before Confirm paid.
- Reject fields: rejected_reason (required, min 8 characters).
- On success: toast, audit log entry, refresh queue, fire notify-payout-status.

12.3 Wiring

adminApi.js already has updateTableRow. Do not use it for status. Add completeWithdrawal(config, payload) and rejectWithdrawal(config, payload) that call supabase.rpc('admin_complete_withdrawal', ...) with the service-role client. Use existing AuditProvider.addLog with entity=withdrawal_requests and the UTR in the payload.

12.4 Filter bugfix (ship with the panel)

PaymentsPage.jsx withdrawals filterFields currently: all, pending, approved, rejected. Change to all, pending, processing, completed, rejected even before the new panel, so today's TEST MODE queue is operable.

Do not let operators edit mentor_wallets.balance in CRUD

Wallets tab should be read-only. A manual balance edit desynchronises total_earned / total_withdrawn and makes F1-style losses undetectable. If a correction is ever needed, add a separate admin_adjust_wallet RPC with a reason code.

13 Mobile app changes

13.1 Payout setup

- Replace Razorpay Route form (legal address, KYC email) with: UPI ID, account holder name, optional bank + IFSC.
- Validate UPI client-side (simple regex) and server-side.
- Success state: “Payouts are ready — withdrawals go to {upi}”.
- Remove copy about Razorpay emailing the mentor for KYC.

13.2 Wallet

- payoutReady = Boolean(storedUpi), not Razorpay kyc_status === 'active'.
- Keep MIN_WITHDRAWAL = 5000 and the existing confirm dialog.
- Load withdrawal_requests for the mentor (web already does this). Show pending / processing / completed / rejected.
- While a request is pending or processing, show it above the withdraw CTA so the mentor does not think the money vanished.
- Copy: “We’ll send this to your UPI from Connectiqo within 1–2 business days.” Do not mention Razorpay payout.

13.3 API comments

Update paymentApi.requestWithdrawal JSDoc from “Trigger a RazorpayX UPI payout” to “Create a manual withdrawal request for admin fulfilment.” Prevents the next developer from resurrecting RazorpayX from a comment.

14 Web portal changes

The web portal already mirrors the app: same Edge Functions, same ₹5,000 floor, and it already lists withdrawal history on /settings/wallet. Changes are smaller than mobile, but must ship in the same release so mentors do not see Razorpay KYC on web and UPI-only on the app.

- /settings/payout — same UPI-first form; drop linked-account fields.
- payoutApi.createLinkedAccount can be renamed savePayoutDetails and pointed at the new function (or the slimmed one).
- getAccountStatus: treat UPI present as ready; do not depend on Razorpay poll.
- Wallet copy: “Processed within 1–2 business days” stays; remove Razorpay payout wording from terms if section 7 currently implies a PSP payout method.
- Terms page currently: “Mentors receive session earnings ... via the payout method configured in Settings.” Keep that; it already fits manual UPI.

15 Notifications and mentor experience

Booking status already has notify-booking-status (FCM). Reuse the same FIREBASE_SERVICE_ACCOUNT pattern for payout events. Suggested payloads:

Event	Title	Body
Request created	Withdrawal requested	₹{amount} to {upi}. We'll send it within 1–2 business days.
Processing	Payout in progress	We're sending ₹{amount} to {upi}.
Completed	Payout sent	₹{amount} sent to {upi}. Ref {utr}.
Rejected	Withdrawal not sent	₹{amount} was returned to your wallet. Reason: {reason}.

Table 11 — Mentor-facing copy. Amounts formatted en-IN.

In-app NotificationsScreen is booking-centric today. For v1, FCM + the Wallet status list is enough. A dedicated notification category can follow if support volume is high.

16 Security, audit, and India compliance

16.1 Application security

- Mentors can only insert their own withdrawal_requests (existing RLS).
- Wallet mutation only inside SECURITY DEFINER RPCs — never from the client.
- Admin complete/reject requires service-role (already how admin mutates).
- Do not accept status updates from the mentor JWT.
- UTR uniqueness optional: warn if the same payout_reference is reused (catch paste errors / double complete).
- Log operator email on every transition (processed_by + AuditProvider).

16.2 Admin session (known limitation)

Admin AuthProvider compares VITE_ADMIN_EMAIL / VITE_ADMIN_PASSWORD in the browser and stores a localStorage flag. The service-role key is also in the admin env and used from the client. That is pre-existing. Payouts make it more sensitive. Phase 2 should move fulfilment RPCs behind an Edge Function that checks an admin secret or a real Supabase admin user, and stop shipping the service-role key to the browser. Do not block v1 on that rewrite if the admin app is already trusted on an internal machine — but do not expand service-role usage into new public clients.

16.3 Operational / finance notes (India)

- Connectiqo is merchant of record for the learner payment; mentor payout is a payable.
- Keep UTR, amount, beneficiary UPI, date — needed for bank reconciliation and disputes.
- TDS (e.g. 194J/194H) may apply above thresholds; v1 should let finance record a note. Auto-deduct only if they specify a rate.
- GST: platform fee is already split at checkout. Mentor invoices (if GST-registered) are a finance process, not an app feature in v1.
- Pay from a company current account / business UPI, not a personal account.
- Daily cap: if one operator would send more than the bank's UPI limit, use IMPS/NEFT and store method=neft.

17 Implementation phases

Phase	Goal	Ships
0 — Stabilise	Make the current TEST MODE safe	Version deduct_wallet_for_withdrawal; fix admin status filter; freeze wallet CRUD; document TEST MODE as intentional.
1 — Fulfil	Operators can pay	New columns; complete/reject RPCs; Payout Control Panel; Copy UPI/amount; notify-payout-status.

Phase	Goal	Ships
2 — Identity	Drop Razorpay KYC from mentor UX	UPI-first payout setup on app + web; get-account-status without Razorpay; disable transfer-session-payout.
3 — Scale (optional)	When queue is large	CSV / NEFT bulk export; maker-checker; admin auth hardening (no service-role in browser).

Table 12 — Phased delivery. Phase 1 unblocks operations even if payout setup is still the old form, because process-withdrawal already only needs `upi_id`.

Suggested order of work in git: one migration PR (RPCs + columns), one admin PR (panel), one app+web PR (setup + wallet list). Backend first so admin can fulfil requests that mentors already create today.

18 Test plan

ID	Case	Expected
T1	Withdraw below ₹5,000	Client rejects; no row; no wallet change.
T2	Withdraw without UPI	Edge error: Complete payout setup first.
T3	Withdraw more than balance	RPC error; no request row.
T4	Happy withdraw	Balance held; pending row; UPI copied onto the row.
T5	Admin complete with UTR	Status completed; total_withdrawn += amount; FCM sent.
T6	Admin complete twice	Second call no-ops / errors; wallet not double-counted.
T7	Admin reject	Status rejected; balance restored; total_withdrawn unchanged; FCM.
T8	CRUD JSON status edit	Blocked or ignored; only RPCs change status.
T9	App and web same mentor	Both show the same pending request and new balance.
T10	Session complete still credits wallet	Unrelated booking still runs complete_session_payment.
T11	Learner Razorpay checkout	Unchanged; regression on booking payment.
T12	Invalid UPI save	Rejected at Edge; payout not ready.

Table 13 — Minimum QA before production fulfilment.

19 What not to do

- Do not uncomment or re-add a RazorpayX payout block in process-withdrawal.
- Do not require Razorpay Route KYC before a mentor can request a withdrawal.
- Do not mark requests completed from the generic JSON editor.
- Do not decrement the wallet again on complete (it is already held on request).
- Do not increment total_withdrawn on request if you also increment on complete.
- Do not pay from a personal UPI and skip the UTR field.
- Do not change learner Razorpay collection as part of this work.
- Do not add an “approved” status unless maker-checker is a real requirement.

Definition of done

A mentor with a saved UPI and ₹5,000+ available can request a withdrawal on app or web. An admin can pay that UPI from the company account, record the UTR, and the mentor's wallet and history match — without any Razorpay payout API call in the logs.

20**Appendix — current schema and file index****A. withdrawal_requests (as created)**

```

id                UUID PK
mentor_id         UUID NOT NULL → profiles
amount            NUMERIC(12,2) NOT NULL
upi_id            TEXT
bank_account       TEXT
status            TEXT DEFAULT 'pending'
                  CHECK (pending | processing | completed | rejected)
admin_note         TEXT
razorpay_payout_id TEXT          -- added later; unused in TEST MODE
created_at         TIMESTAMPTZ
updated_at         TIMESTAMPTZ

```

RLS: mentor SELECT + INSERT own rows. No mentor UPDATE.

B. mentor_wallets

```

id                UUID PK → profiles
balance           NUMERIC(12,2) -- available after holds
total_earned       NUMERIC(12,2)
total_withdrawn    NUMERIC(12,2)
updated_at         TIMESTAMPTZ

```

C. Existing RPCs to keep

- increment_mentor_wallet — used at credit time in some paths.
- complete_session_payment — session complete → earnings completed + wallet credit.
- claim_and_book_slots — atomic booking + pending earnings after Razorpay verify.

D. File index (absolute)

```

C:\Flutter\Freelancing\Project\connectfront\supabase\functions\process-withdrawal\index.ts
C:\Flutter\Freelancing\Project\connectfront\supabase\functions\create-linked-account\index.ts
C:\Flutter\Freelancing\Project\connectfront\supabase\functions\get-account-status\index.ts
C:\Flutter\Freelancing\Project\connectfront\supabase\functions\transfer-session-payout\index.ts
C:\Flutter\Freelancing\Project\connectfront\supabase\migrations\001_payment_tables.sql
C:\Flutter\Freelancing\Project\connectfront\supabase\migrations\002_earnings_status.sql
C:\Flutter\Freelancing\Project\connectfront\supabase\migrations\add_payout_fields.sql
C:\Flutter\Freelancing\Project\connectfront\src\scenes\settings\WalletScreen.js
C:\Flutter\Freelancing\Project\connectfront\src\scenes\settings\PayoutSetupScreen.js
C:\Flutter\Freelancing\Project\connectfront\src\api\paymentApi.js
C:\Flutter\Freelancing\Project\connectfront\src\api\payoutApi.js
D:\connectiqo\connectiqo web\connectiqo web\connectiqo_web-portal\src\app\main\settings\wallet\page.tsx
D:\connectiqo\connectiqo web\connectiqo web\connectiqo_web-portal\src\app\main\settings\payout\page.tsx
D:\connectiqo\connectiqo web\connectiqo web\connectiqo_web-portal\src\lib\api\paymentApi.ts
D:\connectiqo\connectiqo web\connectiqo web\connectiqo_web-portal\src\lib\api\payoutApi.ts
C:\Flutter\Freelancing\Project\admin\adminsideconnectfornt\admin\src\features\payments\PaymentsPage.jsx
C:\Flutter\Freelancing\Project\admin\adminsideconnectfornt\admin\src\features\payments\TransactionControlPanel.jsx
C:\Flutter\Freelancing\Project\admin\adminsideconnectfornt\admin\src\services\adminApi.js

```

End of report. Next concrete build step: Phase 0 migration for deduct_wallet_for_withdrawal plus the admin status-filter fix, then Phase 1 Payout Control Panel.